

Java

Zuletzt aktualisiert am:

27.09.2026

Inhaltsverzeichnis

1. Daten: Bits & Bytes	1
1.1. Datentypen	1
1.2. Primitive Datentypen	1
2. Variablen	2
2.1. Variablen deklarieren und initialisieren	3
3. Ausdrücke	3
3.1. Wahrheitswerte mit Boolean	5
3.2. Logische Verknüpfungen	6
3.2.1. Konjunktion: <code>&&</code>	7
3.2.2. Disjunktion: <code> </code>	7
3.2.3. Negation: <code>!</code>	7
3.2.4. Komplexe Ausdrücke und Präzedenzregeln	8
4. Zeichen & Texte	9
4.1. Stringkonkatenation	10
5. Die Kontrollstruktur <code>if</code>	11
5.1. Wenn ... andernfalls: <code>else</code>	12
5.2. Ein <code>else if</code> ergänzen	13
6. Schleifen	15
6.1. Die <code>while</code> -Schleife	15
6.2. Der Klassiker: die Endlosschleife	16
6.3. Syntax	17
6.4. Iteration	17
6.5. Bedingungen ohne Zählen	18

1. Daten: Bits & Bytes

Computerhardware ist im Grunde Elektrotechnik im Miniformat: Es gibt nur **strom fließt** oder **strom fließt nicht**.

Die Auswirkungen von diesem simplen Modell reicht von der untersten Hardwareschicht bis hoch zu Programmiersprachen wie Java. Software besteht im wesentlichen aus **Daten** und **Anweisungen**. Daten werden in **0**en und **1**en dargestellt, auch **Bits** genannt, und Anweisungen schubsen diese Bits nur hin und her.

Durch die Hardwarearchitektur des *Hauptspeichers* kommt ein Bit selten allein:

Ein **Byte** besteht aus **8 Bits**.

Durch 1 Byte (bestehend aus 8 Bits) lassen sich durch das Binärsystem $2^8 = 256$ verschiedene Zustände erfassen. Durch Kombination von mehreren Bytes können wir die darstellbaren Zustände noch erweitern. Diese nutzen wir, um komplexere Daten darzustellen, wie Dezimalzahlen, Kommazahlen und Text.

Je nachdem, was für Informationen dargestellt werden sollen, werden die zugehörigen Bits und Bytes unterschiedlich verarbeitet. Zur Darstellung von Text werden z. B. Zeichensatztabellen verwendet, um einen Buchstaben einer Binärzahl zuzuordnen.

1.1. Datentypen

Um solche Bytehaufen besser zu handhaben, definieren Programmiersprachen feste **Datentypen** die unsere ersten Grundbausteine (über den Bytes) bilden.

Was sind Datentypen?

Ein Datentyp gibt an:

- von welcher Art die Daten sind, die mit ihm beschrieben werden (Datenvereinbarung)
- wie viele Bytes an Speicherplatz der Typ benötigt
- welche Operationen/Anweisungen mit diesem Typen möglich sind

Beispiel am Java Datentypen `int`:

- er ist eine Ganzzahl
- er kann Zahlenwerte von $-2.147.483.648$ bis $2.147.483.647$ speichern
- mit dem Wert kann gerechnet werden
- der Wert kann mit einem anderen verglichen werden
- der Wert benötigt 4 Byte bzw. 32 Bit Speicherplatz

1.2. Primitive Datentypen

Primitive Datentypen, auch einfache Datentypen genannt, bilden immer nur ihren Wert ab. Primitive Datentypen sind »primitiv«, weil sie im Gegensatz zu komplexen Datentypen keine weitere Funktionalität haben. Alle komplexen Datentypen bauen auf den primitiven Datentypen auf.

Auf komplexe Datentypen wird aber erst später eingegangen.

Zahlen-Datentypen in Java sind immer **signed**, d.h. sie teilen den negativen und positiven Wertebereich auf. Es gibt in Java 8 primitive Datentypen:

Name	Art der Daten	Wertbereich	Speicherbedarf
<code>byte</code>	Ganzzahl	–128 bis 127	1 Byte
<code>short</code>	Ganzzahl	–32.768 bis 32.767	2 Byte
<code>int</code>	Ganzzahl	–2.147.483.648 bis 2.147.483.647	4 Byte
<code>long</code>	Ganzzahl	–9.223.372.036.854.775.808 bis 9.223.372.036.854.775.807	8 Byte
<code>float</code>	Gleitkommazahl	- Einfache Genauigkeit - ca. 6-7 Nachkommastellen - große Zahlen: $3,402823466 \cdot 10^{38}$ - kleine Zahlen: $1,401298464 \cdot 10^{45}$	4 Byte
<code>double</code>	Gleitkommazahl	- Doppelte Genauigkeit - ca. 12-14 Nachkommastellen - große Zahlen: $\pm 1,797693134862 \cdot 10^{308}$ - kleine Zahlen: $\pm 4,940656458412 \cdot 10^{-324}$	8 Byte
<code>char</code>	Zeichen	0 bis 65.535 (UTF-16 / Unicode)	2 Byte
<code>boolean</code>	Wahrheitswert	<code>true</code> oder <code>false</code>	1 Bit (theoretisch), 1 Byte in der Praxis

Tabelle 1: Primitive Datentypen in Java

Die drei markierten Datentypen werden Sie am meisten im Programmieralltag verwenden.

Gleitkommazahlen (Fließkommazahlen) wie `float` haben durch ihre limitierte, binäre Darstellung im Speicher komplexe und unintuitive¹ Rechengesamkeiten. Daher ist die Anzahl an »Nachkommastellen« oder der »Wertebereich« der darstellbaren Zahlen nur bedingt hilfreich.

In der [IEEE 754](https://de.wikipedia.org/wiki/IEEE_754)² Spezifikation ist die Funktionsweise genau definiert. Für die Alltagsgebrauch ist ein ausgiebiges Studium der Spezifikation zum Glück nicht notwendig.

2. Variablen

Variablen sind benannte Speicherstellen im Hauptspeicher eines Rechners. Mit ihrer Hilfe können wir uns Daten (Werte) für eine spätere Verwendung speichern. Sie sind damit das meistverwendete Konstrukt in Programmiersprachen.

In streng typisierten Hochsprachen wie Java muss bei der Verwendung von Variablen immer angegeben werden, welchen Datentyp die Variable hat. Eine Variable hat immer genau einen Datentyp, der nachträglich nicht mehr geändert werden kann.

¹Addieren Sie später z. B. mal die Zahlen `0.1` und `0.2`. Das Ergebnis wird Sie vermutlich etwas enttäuschen.

²https://de.wikipedia.org/wiki/IEEE_754

2.1. Variablen deklarieren und initialisieren

Variablen werden:

1. **dekliert:** Der *Datentyp* und der *Variablenname* wird angegeben. Der Name³ ist frei wählbar und wird genutzt, um die Variable später weiterverwenden zu können.

Java

```
// Datentyp  Bezeichner  Anweisungsende
int         number    ;
```

2. anschließend **initialisiert:** Zuweisung eines gültigen Wertes

Java

```
// Bezeichner  Zuweisungsoperator  Wert  Anweisungsende
number        =                    6    ;
```

Beides kann auch in einem Schritt erfolgen.

Java

```
int number = 6;
```

Übung 1



Deklarieren und initialisieren Sie **alle primitiven, numerischen Datentypen** in Java mit passenden Werten ihrer Wahl (sechs Typen insgesamt). Siehe Tabelle 1.

Beachten Sie:

Werte des Typs `float` werden zwingend mit einem `f` unmittelbar hinter der Zahl markiert, und Werte des Typs `long` mit einem `L`.

Jede Anweisung schließt mit dem Semikolon ab.

3. Ausdrücke

Bevor einer Variable ein Wert mit dem `=` zugewiesen wird, können Sie diesen Wert vorher mit einem sog. **Ausdruck** ermitteln. Ein Ausdruck kann z. B. eine arithmetische Rechnung sein.

Operator	Bedeutung	Zeichen
+	Addition	Plus
-	Subtraktion	Minus
*	Multiplikation	Sternchen
/	Division	Schrägstrich
%	Modulo bzw. Rest einer Division	Prozent

Tabelle 2: Arithmetische Operatoren in Java

Welchen Wert hat die Variable `result`?

Java

```
int result = (5 + 3) * 2;
//          -----  1. der Ausdruck '5 + 3' wird evaluiert
//          -----  2. dann der Ausdruck '... * 2'
//  -----  3. als letztes 'result = ...' (Zuweisung)
```

³Der Variablenname ist in der Fachsprache ein *Bezeichner* (Eng. *Identifier*).

Diese drei geschachtelten Ausdrücke (expressions) bilden zusammen eine Anweisung (statement).⁴

Bei der Bildung von Ausdrücken muss die Syntax von Java berücksichtigt werden, die der klassischen Algebra sehr ähnlich ist aber nicht immer identisch! Auch in Java gilt z. B. Punkt- vor Strichrechnung.

Ausblick

Ausdrücke können auch an vielen anderen Stellen in Java's Syntax verwendet werden. Nicht nur bei der Initialisierung von Variablen. Behalten Sie das im Hinterkopf, wenn Sie im Verlauf des Unterrichts weitere Java-Features kennenlernen!

In einem Ausdruck können wir bereits deklarierte **Variablen** wiederverwenden. Welchen Wert hat die Variable `doubleResult`?

Java

```
int number1 = 1;
int number2 = 2;
int result = number1 + number2;
int doubleResult = result * 2;
```

Java-Konvention für Variablennamen

Variablennamen dürfen keine Leerzeichen enthalten und nicht mit einer Ziffer beginnen.

Nach *Konvention* starten Variablennamen mit einem kleinen Buchstaben. Für zusammenhängende Begriffe verwenden wir **camelCase**:⁵ kleiner Anfangsbuchstabe, neue Wörter durch Großbuchstaben trennen.

Übung 2

Geben Sie für die markierten Variablen den Wert an:

Java

```
int a = 3;
int b = 5;

int c = a * b - 2; // Wert von c ?
int d = b / a;     // Wert von d ?
int e = c;         // Wert von e ?

a = a + 1;         // Wert von a ? (das linke a)
```

⁴Siehe auch: <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/expressions.html>

⁵In anderen Teams oder Projekten wird (seltener) auch *snake_case* für Variablen verwendet. Hier werden *alle* Buchstaben klein geschrieben und Wörter werden durch `_` voneinander getrennt.

Ausgabe auf der Konsole

Sie können sich Werte von Variablen in der *Konsole* ausgeben lassen:

java

```
int myVariable = 12;
IO.println(myVariable);
```

Klicken Sie auf den grünen ▶ Button oder drücken Sie die beiden Tasten **Shift** + **F10** gleichzeitig. Ihr Ergebnis wird in einem neuen Konsolenfenster ausgegeben.

3.1. Wahrheitswerte mit Boolean

Werte von Variablen gleichen Typs können mithilfe von *Vergleichsoperatoren* miteinander verglichen werden. Das Ergebnis eines Vergleichs ist immer ein Wahrheitswert (**boolean**), der aussagt, ob der Vergleich wahr (**true**) oder unwahr (**false**) ist.

8 und 7 sind in diesem Beispiel *Integer* und können verglichen werden:

Java

```
boolean myBool = 8 > 7;
// 8 ist größer als 7
// myBool hat den Wert TRUE
```

Mit Vergleichsoperatoren lassen sich alle möglichen Vergleiche anstellen:

Operator	Bedeutung
==	gleich
!=	ungleich
>	größer als
<	kleiner als
>=	größer oder gleich
<=	kleiner oder gleich

Tabelle 3: Vergleichsoperatoren in Java

Java

```
int age = 17;
boolean isAdult = age >= 18;
```

Wert von **isAdult** ist **false**, da 17 nicht größer oder gleich 18 ist.

Java

```
int calculation = (3 + 9) / 4;
int solution    = 3;

boolean correct = calculation == solution;
```

Wert von **correct** ist **true**, da 3 gleich 3 ist.

Verwechslungsgefahr!



Verwechseln Sie nicht **=** mit **==** !

Das **=** ist der *Zuweisungsoperator* für Variablen, das **==** ist ein *Vergleichsoperator*.

Übung 3



Geben Sie für die markierten Zeilen jeweils an, ob der Vergleich wahr oder unwahr ist:

Java

```
int    day        = 1;
int    month      = 4;
boolean isNiceWeather = true;
boolean isWeekend  = false;

// Welche Werte haben die nachfolgenden Variablen?
boolean a          = 5 <= 5;           // (a)
boolean b          = 9 == 8;           // (b)
boolean notFirstDay = day > 1;          // (c)
boolean isSeptember = month == 9;      // (d)
boolean free        = isWeekend == true; // (e)
boolean laterThanAugust = month > 8;    // (f)
```

Übung 4



Schreiben Sie selbst einen beliebigen Ausdruck, in dem Sie einen `int` mit einem `boolean` vergleichen. Was beobachten Sie?

3.2. Logische Verknüpfungen

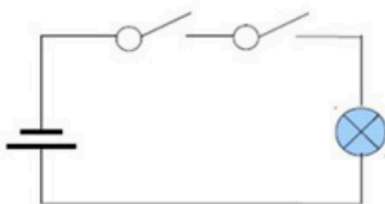
Wichtiger Bestandteil der *booleschen Algebra* ist die Verknüpfung mehrerer Aussagen. Wir behandeln an dieser Stelle erstmal folgende Operatoren:

Fachbegriff	Operator	Bedeutung
Konjunktion	&&	UND
Disjunktion		ODER
Negation	!	NICHT

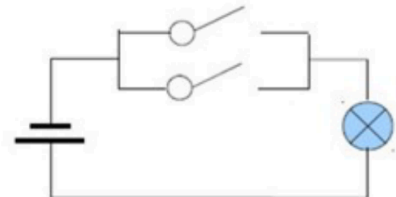
Tabelle 4: Boolesche Verknüpfungen

Mit dem Schaltbild eines elektrischen Schaltkreises lässt sich die Funktionsweise von UND und ODER gut erkennen. Die UND-Verknüpfung entspricht der Reihenschaltung. DIE ODER-Verknüpfung entspricht der Parallelschaltung.

UND



ODER



3.2.1. Konjunktion: &&

Die Gesamtbedingung ist nur wahr, wenn **alle** Teilbedingungen wahr sind.

java

Beispiel 1: `boolean someBoolean = 5 == 5 && 7 < 9 && 10 > 8;`

Alle drei Teilbedingungen sind wahr, also ist die Gesamtbedingung (`someBoolean`) wahr.

java

Beispiel 2: `boolean someOtherBoolean = 5 == 5 && 7 != 7;`

Die zweite Teilbedingung ist nicht wahr, also ist die Gesamtbedingung (`someOtherBoolean`) falsch. Das logische Verhalten der Konjunktion lässt sich in einer **Wahrheitstabelle** veranschaulichen:

Teilbedingung 1	Teilbedingung 2	Gesamtbedingung
wahr ✓	wahr ✓	wahr ✓
wahr ✓	falsch ✗	falsch ✗
falsch ✗	wahr ✓	falsch ✗
falsch ✗	falsch ✗	falsch ✗

Tabelle 5: Wahrheitstabelle für Beispiel 2

3.2.2. Disjunktion: ||

Die Gesamtbedingung ist wahr, wenn **mindestens eine** Teilbedingung wahr ist.

java

Beispiel 1: `boolean myBoolean1 = 5 == 5 || 7 != 7;`

Einer der Teilbedingungen ist wahr, also ist die Gesamtbedingung (`myBoolean1`) wahr.

java

Beispiel 2: `boolean myBoolean2 = 5 == 5 || 7 < 9 || 10 > 8;`

Alle drei Teilbedingungen sind wahr, also ist die Gesamtbedingung (`myBoolean2`) wahr.

java

Beispiel 3: `boolean myBoolean3 = 3 < 2 || 7 != 7;`

Keine Teilbedingung ist wahr, also ist die Gesamtbedingung (`myBoolean3`) falsch.

3.2.3. Negation: !

Die Negation dreht den Wahrheitswert in sein Gegenteil um, sie **negiert** ihn. Aus wahr wird falsch und umgekehrt.

java

Beispiel 1: `boolean example = !(5 == 5);`

Der Wahrheitswert von `5 == 5` ist wahr, die Negation davon (`example`) ist also falsch. Die Klammern sind hier wichtig, um den gesamten Wahrheitswert zu negieren.

java

Beispiel 2: `boolean rainy = false;
rainy = !rainy;`

In diesem Code Snippet überschreiben wir die Variable `rainy` mit ihrer Negation. Da sie vorher `false` war, ist sie nun `true`.

3.2.4. Komplexe Ausdrücke und Präzedenzregeln

Es ist möglich, beliebig viele logische Ausdrücke mit UND/ODER zu verknüpfen und mit NICHT zu negieren.

Dabei gibt es Präzedenzregeln, in welcher Reihenfolge solche Ausdrücke aufgelöst und zu einem Wahrheitswert kombiniert werden. Genau wie in der Arithmetik Punkt- vor Strichrechnung gilt, gilt in logischen Ausdrücken UND vor ODER.

Man sagt: Der UND-Operator hat eine **höhere Präzedenz** als der ODER-Operator.

Die Präzedenzregeln der logischen Operatoren sind folgende:

1. Negationen werden zuerst aufgelöst
2. Dann werden UND-Verknüpfungen aufgelöst
3. Danach werden die ODER-Verknüpfungen aufgelöst
4. Zum Schluss wird grundsätzlich von links nach rechts aufgelöst

Diese Präzedenzregeln lassen sich mit Klammern genau wie in der Arithmetik übergehen. Als Beispiel für die Verknüpfung von logischen Ausdrücken setzen wir folgende zwei Aussagen in Java um:

- **WENN** heute schönes Wetter **UND NICHT** Unterricht findet statt **Dann** gehe ich spazieren
- **WENN** heute schönes Wetter **ODER** Unterricht findet statt **Dann** verlasse ich das Haus

java

```
boolean isNiceWeather = true;
boolean isSchool      = true;

boolean goWalk;
boolean leaveHouse;

//      true      && !true      => false
goWalk = isNiceWeather && !isSchool;

//      true      || true      => true
leaveHouse = isNiceWeather || isSchool;
```

Übung 5



Schreiben Sie einen Vergleich nach dem obigen Prinzip für folgende Fragestellung:

java

```
double speed = 50;
...
boolean speedIsLegal = ...;
IO.println(speedIsLegal);
```

WENN Geschwindigkeit == 50 UND im Ort UND NICHT 30er-Zone DANN Geschwindigkeit eingehalten (`speedIsLegal = true`).

Ob der Fahrer im Ort ist, und/oder in einer 30er-Zone ist, soll durch `boolean` festgelegt werden können (wie die Geschwindigkeit im Beispiel). Testen Sie Ihre Verknüpfung mit unterschiedlichen Werten.

Übung 6



Schreiben Sie auf, welcher Wahrheitswert aus den jeweiligen Verknüpfungen entsteht. Beachten Sie dabei die Präzedenzregeln.

java

```
int    a = 9;
float  b = 2.5f;
boolean c = false;
boolean d = true;

boolean e = a == 9 || b > 3f;    // Wert von e ?
boolean f = a > 5 && c;          // Wert von f ?
boolean g = 5 > 5 && 10 == 9;    // Wert von g ?
boolean h = b != 3f || d && !c;  // Wert von h ?
boolean i = (b == 3f || d) && !c; // Wert von i ?
```

Übung 7



Finden Sie die falsch angegebenen Wahrheitswerte.

java

```
boolean a = !(5 > 6) && false;    // true
boolean b = false && false || true // false
boolean c = true && (10 == 9) || false // false
boolean d = (false || true) && true // false
```

4. Zeichen & Texte

Bisher haben wir uns mit Datentypen für Zahlen sowie für Wahrheitswerte beschäftigt. Nun kommen Datentypen für Zeichen (`char`) und Zeichenketten (`String`) hinzu. Die Unterscheidung ist wichtig.

Der Datentyp `char`:

- wird genutzt, um ein einzelnes Zeichen zu speichern
- ist ein **primitiver** Datentyp
- `char` wird in Java klein geschrieben, weil es ein primitiver Typ ist
- Werte von `char` werden in Java in einfachen Anführungszeichen geschrieben: `'a'`

Der Datentyp `String`:

- speichert ganze Zeichenketten (Texte)
- die Anzahl an nötigen Bytes im Speicher variiert je nach Textlänge
- ist kein primitiver, sondern ein **komplexer** Datentyp
- bringt daher zahlreiche Funktionen zur Verwendung von Zeichenketten mit
- `String` wird in Java groß geschrieben, weil es ein komplexer Typ ist
- String-Werte werden in Java in doppelten Anführungszeichen geschrieben: `"Hallo"`

java

```
char    mySingleLetter    = 'a';
String  myStringOfLetters = "Java";
String  password          = "3%$!!pX&45"; // Beliebige UNICODE Zeichen sind möglich
String  thumbsUp          = "Thumbs up: 👍"; // ... auch Emojis
```

4.1. Stringkonkatenation

Mehrere Zeichenketten (also Strings) können miteinander verbunden werden, indem der `+` Operator verwendet wird. Er dient hierbei nicht als arithmetischer Operator, sondern als **Konkatenations-Operator**.

Genauso wie Zeichenketten können auch primitive Datentypen (mit `+`) an Strings angefügt werden. Diese werden dabei implizit in String umgewandelt.

Beispiel:

java

```
int    age    = 12;
String name    = "Uwe";
String greeting = "Hallo " + name + "! Du bist " + age + " Jahre alt.";
```

Übung 8



Fügen Sie die folgenden Variablen mit Hilfs-Strings so zusammen, sodass dieser Satz herauskommt:

Ines ist 18 Jahre alt, Karin ist 32 Jahre alt, Jochen ist 29 Jahre alt.

java

```
String name1 = "Ines";
String name2 = "Karin";
String name3 = "Jochen";

String yearsOld = "Jahre alt";

int age1 = 18;
int age2 = 32;
int age3 = 29;
```

Legen Sie eine neue String-Variable an, wenn nötig.

String ist ein *komplexer* Datentyp



`String` ist als Datentyp eine sog. *Klasse* aus der *Objektorientierten Programmierung* die wir später genauer kennenlernen werden. Geben Sie sich aber ruhig mal folgende Variablen in der Konsole aus:

java

```
String name = "Thomas";
String test = "";

int nameLength = name.length();
int testLength = test.length();

boolean nameIsEmpty = name.isEmpty();
boolean testIsEmpty = test.isEmpty();
```

Gibt `.length()` wirklich die Länge der Zeichenkette aus?

Was passiert, wenn Sie Emojis verwenden (Windowstaste + `.`)?

5. Die Kontrollstruktur `if`

Kehren wir an dieser Stelle zurück zu den *Ausdrücken* und dem Datentyp `boolean`. Das nächste Java-Sprachkonstrukt, das wir kennenlernen werden, nutzt boolesche Ausdrücke, um den Ablauf eines Programms anhand von Bedingungen zu **verzweigen**. Das bedeutet, dass unter einer bestimmten Bedingung etwas passieren soll und sonst nicht.

Dazu nutzen wir die **Kontrollstruktur `if`**, die es in den meisten Programmiersprachen gibt. Man spricht auch von einem **if-Statement**. Die allgemeine Syntax eines if-Statements ist folgende:

```

if (Hier eine Bedingung als Boolean)
{
    Hier Code, der nur im Fall der obigen Bedingung ausgeführt werden soll;
    Dieser Code steht in einem sog. "if-Block", den man an den { ... } erkennt;
    ...
}
  
```

Ein if-Statement benötigt immer eine **Bedingung**, die in runden Klammern `(...)` hinter dem Schlüsselwort `if` folgt. Im **if-Block** steht Code in geschweiften Klammern `{ ... }`, der ausgeführt werden soll. Der Code im if-Block wird nur dann ausgeführt, wenn die Bedingung den Wahrheitswert `true` hat. Gilt die Bedingung nicht, wird der if-Block nicht ausgeführt.

Nur boolesche Ausdrücke erlaubt!



Eine if-Bedingung **muss immer** zu einem Wahrheitswert (`true` oder `false`) evaluiert werden können.

Zwei Beispiele in Java:

java

```

int age = 18;

if (age >= 18)
{
    IO.println("Die Person ist volljährig.");
}
  
```

java

```

boolean isSunday = true;

if (isSunday == true)
{
    IO.println("Es ist Sonntag.");
}

// Das "== true" können Sie weglassen
if (isSunday)
{
    IO.println("Es ist Sonntag.");
}
  
```

In den runden Klammern kann ein beliebig komplex verknüpfter boolescher Ausdruck stehen. Testen Sie z. B. die booleschen Ausdrücke aus den vorherigen Kapiteln und Übungen.

Übung 9



Legen Sie eine `float`-Variable `price` mit dem Wert 9,99 an. Schreiben Sie ein `if`-Statement: Wenn der Wert von `price` gleich 0,0 ist, soll auf die Konsole ausgegeben werden: »Artikel ist kostenlos!«.

5.1. Wenn ... andernfalls: `else`

Bisher können wir angeben, was passieren soll, wenn eine bestimmte Bedingung wahr ist.

Das Schlüsselwort `else` kann hinter einem `if`-Statement ergänzt werden. So kann angegeben werden, was passieren soll, wenn die `if`-Bedingung **nicht wahr** ist.

Die allgemeine Syntax von `if` und `else` zusammen sieht folgendermaßen aus:

text

```
if (Bedingung)
{
    Dieser if-Block wird bei 'Bedingung == true' ausgeführt;
    ...
}
else
{
    Dieser else-Block wird andernfalls ausgeführt;
    ...
}
```

Ein Beispiel in Java:

java

```
int age = 16;
boolean isAdult;

if (age >= 18)
{
    isAdult = true;
}
else
{
    isAdult = false;
}

// Ausgabe des Wertes von isAdult auf Konsole:
IO.println(isAdult);
```

Übung 10



Legen Sie zwei Variablen an:

- Eine `int`-Variable `februaryDays` ohne Wert (Deklaration)
- Eine `boolean`-Variable `isLeapYear` mit dem Wert `true`

Schreiben Sie ein `if/else`-Statement: Wenn die Variable `isLeapYear` den Wert `true` hat, soll die Variable `februaryDays` den Wert 29 bekommen. Sonst soll sie den Wert 28 bekommen.

Zum Schluss geben Sie den Wert von `februaryDays` auf die Konsole aus.

5.2. Ein `else if` ergänzen

Häufig möchte man nicht nur eine Bedingung prüfen, sondern mehrere. Mit dem Schlüsselwort `else if` können nach einer `if`-Bedingung eine oder mehrere alternative Bedingungen definiert und geprüft werden:

java

```
int age = 16;

if (age < 12)
{
    IO.println("Du darfst nicht rein!")
}
else if (age < 16)
{
    IO.println("Du darfst in Begleitung eines Erwachsenen rein.")
}
else if (age < 18)
{
    IO.println("Du darfst bis 22 Uhr rein.")
}
else
{
    IO.println("Du darfst rein!")
}
```

Ein `else if`-Statement muss sich immer hinter einem `if`-Statement befinden und vor einem optionalen `else`-Statement. Das `else`-Statement darf es nur *einmal* geben und befindet sich immer ganz am Ende der bedingten Verzweigung.

Beim Ergänzen von `else if`-Statements wird die Reihenfolge der Bedingungsabfragen sehr wichtig: Sobald eine der geprüften Bedingungen `true` ist, wird **keiner der nachfolgenden** `else if`-Blöcke ausgeführt, selbst wenn diese auch `true` sein sollten.

Betrachten Sie dazu folgendes **fehlerhaftes** Beispiel:

java

```
int a = 3;

if (a < 5)
{
    IO.println("a ist kleiner als 5")
}
else if (a < 4)
{
    // Dieser Block wird nicht mehr ausgeführt, obwohl die Bedingung wahr ist
    IO.println("a ist kleiner als 4")
}

// Konsolenausgabe ist: "a ist kleiner als 5"
```

Übung 11



In den folgenden Code haben sich **drei Fehler** eingeschlichen. Identifizieren und korrigieren Sie diese:

java

```
int a = 3;

if (3)
{
    IO.println("a ist 3!");
}
else (a > 3)
{
    IO.println("a ist größer als 3!");
}
else if (a < 3)
{
    IO.println("a ist kleiner als 3!");
}
```

6. Schleifen

Beim Programmieren gibt es häufig Situationen, in denen eine Anweisung mehrfach ausgeführt werden soll. Stellen wir uns beispielsweise vor, wir möchten die Zahlen von 0 bis 9 ausgeben:

java

```
I0.println(0);
I0.println(1);
I0.println(2);
...
I0.println(8);
I0.println(9);
```

Das funktioniert, ist aber umständlich. Noch umständlicher würde es, wenn wir eine Anweisung nicht nur 10 mal, sondern 100 oder 10.000 mal ausführen wollen.

Als Lösung hierfür gibt es **Schleifen**. Schleifen sind genau wie `if` und `else` eine **Kontrollstruktur**. Man benutzt sie, um bestimmte Programmabschnitte wiederholt auszuführen. Der Code wird also nicht einfach Zeile für Zeile von oben nach unten ausgeführt: Codezeilen, die sich in Schleifen befindet, werden unter bestimmten Bedingungen mehrfach ausgeführt.

6.1. Die `while`-Schleife

Um unser Problem aus der Einführung zu lösen, wäre folgender Ansatz sinnvoll:

```
Eine Zahl ist gleich 0

Solange die Zahl kleiner als 10 ist:
{
    Gib die Zahl aus
    Erhöhe die Zahl um 1
}
```

Das entscheidende Stichwort ist hier das Wort *solange* (engl. *while*): Wir führen die eingerückten Anweisungen aus, solange eine Bedingung erfüllt ist.

Die Schreibweise der `while`-Schleife ist in Java der eines `if`-Statements sehr ähnlich. Bei einem `if` wird eine Bedingung überprüft und der zugehörige Code ausgeführt, wenn die Bedingung `true` ist:

java

```
int number = 0;
if (number < 10)
{
    I0.println(number);
    number++;           // äquivalent zu: number = number + 1;
}
```

Kurze Randnotiz: Kurzschreibweise zum Inkrementieren

Das `number++` erhöht den Wert der Variable um 1 und ist äquivalent zu `number = number + 1`.

Diesen sog. *Inkrement-Operator* gibt es auch schon in der Programmiersprache C. Jetzt wird Ihnen vielleicht klarer, warum die Bjarne Stroustrup seine Programmiersprache, die er auf C aufbaute, C++ getauft hat.

Das obige Programm gibt leider nur eine 0 aus, addiert 1 auf `number` und beendet sich noch bevor die nächste Zahl ausgegeben wird. Ersetzen wir aber das Schlüsselwort `if` durch `while`, werden die Zahlen 0 bis 9 ausgegeben.

java

```
int number = 0;
while (number < 10)
{
    IO.println(number);
    number++;
}
```

`if`: Wenn die Bedingung wahr ist, führe die Anweisung(en) aus.

`while`: **Solange** die Bedingung wahr ist, führe die Anweisung(en) aus.

Eine `while`-Schleife kann vereinfacht als ein „wiederholtes if“ verstanden werden.

6.2. Der Klassiker: die Endlosschleife

`while`-Schleifen sind notorisch dafür auch von erfahrenen Softwareentwicklern aus Versehen in Endlosschleifen verwandelt zu werden. Wenn Sie sich trauen, entfernen Sie mal die Zeile `number++` aus dem obigen Codebeispiel und führen Sie das Programm z. B. in IntelliJ aus.

Wie komme ich hier raus?



Theoretisch bricht eine Endlosschleife niemals von selbst ab. Praktisch ist ihre Laufzeit durch einen endlich großen Arbeitsspeicher begrenzt.

Früher war das Ziehen des Steckers oder geduldig warten bis der Computer abstürzt eine Lösung für das Problem. Heutzutage kümmert sich das Betriebssystem um das problematische Programm. Es wird kreidebleich und ein Popup mit der Meldung »Das Programm reagiert nicht mehr ...« gibt Ihnen die Möglichkeit der App den Gnadenschuss zu verpassen.

Ein Griff in den Taskmanager mit der Option »Task beenden« tut es auch.

Häufig werden Endlosschleifen unbeabsichtigt geschrieben aber es gibt Endlosschleifen, die einen konkreten Zweck erfüllen. Z. B. Apps mit grafischen Oberflächen oder Videospiele sind im Kern eine große Endlosschleife, die erst abbricht, wenn das Programm beendet wird. Endlosschleifen sind geeignet dafür auf Ereignisse zu *warten* (z. B. Nutzereingaben). Solche Schleifen sind jedoch gut durchdacht, um nicht die CPU oder den Arbeitsspeicher des Rechners zu belasten.

Übung 12



Passen Sie die `while`-Schleife so an, dass von 9 bis 0 **runtergezählt** wird.

Gibt es einen *Dekrement*-Operator `number--` ? Gibt es auch andere Kurzschreibweisen?

Testen Sie z. B. `number -= 2` oder `number *= 2` !

6.3. Syntax

```
while (Bedingung)
{
    - ... Ihr Code ...
    - Ein "update" für die Bedingung, um Endlosschleifen zu vermeiden
}
```

Eine `while`-Schleife enthält immer die Komponenten *Bedingung* und *Update*:

1. **Bedingung:** Der Code in der Schleife wird ausgeführt, solange diese Bedingung den Wert `true` hat. Wenn diese Bedingung den Wert `false` hat, bricht die Schleife ab, und der Code nach der Schleife wird fortgesetzt. Das Bedingungs-Statement muss immer ein **boolescher Ausdruck** sein.
2. **Update:** Eine Anweisung, die irgendwann dafür sorgt, dass die Bedingung den Wert `false` hat und dadurch die Schleife beendet.
 - Häufig ist dies das Inkrement (Hochzählen) oder Dekrement (Runterzählen) einer Variable, die in der Bedingung geprüft wird. Eine solche Variable, die sich mit jedem Schleifendurchlauf systematisch verändert, nennt sich **Laufvariable**.
 - Meistens befindet sich das Update am Ende des Schleifenblocks, dies muss aber nicht unbedingt der Fall sein.

6.4. Iteration

Gerade bei der Fehlersuche ist es nicht immer einfach den Programmablauf einer Schleife im Kopf durchzuführen. Hier ist es hilfreich die Entwicklung der Laufvariable für jeden Durchlauf der Schleife zu notieren.

Folgender Code gibt uns die ersten vier Zahlen der Siebenerreihe aus:

java

```
int x = 0;
while (x <= 4)
{
    IO.println(7 * x);
    x++;
}
```

Der exakte Ablauf der Schleife lässt sich mithilfe einer Tabelle darstellen, die pro Schleifendurchlauf eine Zeile hat:

Durchlauf Nr.	Wert von x	Bedingung (Ausdruck)	Bedingung	Konsolenausgabe
1	0	0 <= 4	true	0
2	1	1 <= 4	true	7
3	2	2 <= 4	true	14
4	3	3 <= 4	true	21
5	4	4 <= 4	true	28
6	5	5 <= 4	false	-

Durchlauf Nr. 6 findet nicht statt.

Übung 13



Schreiben Sie eine `while`-Schleife, die ersten fünf 2er-Potenzen ausgibt. Die Ausgabe soll also sein: 2, 4, 8, 16, 32.

Schreiben Sie dann eine Durchlauftabelle nach obigem Vorbild.

Tipp: Verdoppeln Sie die Ausgabe mit dem Multiplikationsoperator `*`.

6.5. Bedingungen ohne Zählen

Eine `while`-Schleife muss nicht eine Zahl hoch- oder runterzählen. Die Bedingung kann jede logisch passende Bedingung sein. Das einzige Kriterium ist, dass sie sich zu einem Wahrheitswert evaluieren lässt.

Übung 14



1. Lesen Sie sich den folgenden Code in Ruhe durch und versuchen Sie, ihn nachzuvollziehen. Stellen Sie Vermutungen auf:
 - was die Bedingung in Zeile 3 überprüft
 - was der allgemeine Zweck des Codes ist
2. Führen Sie den Code nun aus, bei Bedarf mehrfach. Vollziehen Sie nach, was in Zeile 4 passiert.

java

```
String input = "";

while (!input.equals("stop")){
    IO.println("Hallo User!");
    input = IO.readLine("Schreibe 'stop' zum Beenden des Programms: ");
}

IO.println("Programm beendet.");
```

Nutzereingaben



Bisher haben unsere Programme in keiner Weise mit dem Nutzer interagiert. Unsere Übungsprogramme starten, machen Ihre Ausgabe und beenden sich. Um Ihr Programm mit unterschiedlichen Werten für Ihre Variablen zu testen, mussten Sie den Code anpassen und es neu kompilieren.

Eine sehr simple Form der Nutzerinteraktion ist eine Eingabeaufforderung. Mit der Funktion `readln()` können Sie den User auffordern eine Eingabe zu machen. Das Programm wartet bis der User mit `Enter` die Eingabe bestätigt und gibt diese als `String` zurück.

Die Eingabe erfolgt in der selben Konsole, in der wir unsere Ausgaben erzeugt haben. Testen Sie folgendes Programm:

Java

```
String name = IO.readLine("Geben Sie Ihren Namen ein: ");
IO.println("Hallo " + name);
```